

JobAgent

Algorithm Reference

Discovery, deduplication, matching & decision logic, end to end

Covers: `scripts/discover_companies.py`
`scripts/discover_jobs.py`
`scripts/score_job.py` (V2 matching engine)

Prepared: 2026-09-13

1. End-to-End Pipeline

Three deterministic, LLM-free algorithms run in sequence. Nothing in this pipeline calls a language model -- discovery, deduplication, normalization, and scoring are all plain rule-based code, which is what makes every decision reproducible and explainable.

STAGE 0	STAGE 1	STAGE 2	STAGE 3
Company Discovery (discover_ companies.py)	Job Discovery & Normalization (discover_jobs.py fetch/normalize)	Deduplication & History Merge (discover_jobs.py identity index)	V2 Matching & Decision (score_job.py)
-->	-->	-->	
v	v	v	v
data/discovery_ sources.json (verified company list grows)	raw normalized job dicts (one common schema)	data/jobs.json (merged, with first_seen_at / last_seen_at)	score, role_family, career_level, application_decision (AUTO_APPLY/REVIEW/SKIP)

Design invariants that hold across every stage

- High recall upstream, precision downstream: nothing before Stage 3 ever rejects a posting for missing salary/experience/location/date -- only the scorer decides fit.
- Nothing is ever guessed into a trusted state: a company board or a job field is only recorded once a real, live response confirms it.
- Every stage is idempotent: re-running the whole pipeline on the same data updates timestamps, never duplicates records.

2. Stage 0 -- Company Discovery & Verification

Input: a maintained, human-curated seed list of candidate company identifiers per ATS connector (data/discovery_candidates.json) -- real company names guessed to run their careers page on a given platform. Output: data/discovery_sources.json gains only the candidates that are confirmed live AND currently carry at least one relevant-sounding opening.

Algorithm (scripts/discover_companies.py)

```
for each connector in {greenhouse, lever, ashby}:
    keywords = expand_target_titles(candidate.target_titles, candidate.secondary_titles)
    checked = 0
    for identifier in candidates[connector]:
        if identifier already in discovery_sources[connector]:
            skip (already_active += 1)
        if checked >= MAX_COMPANIES_PER_SOURCE:
            break
        response = single_http_get(connector, identifier) # ONE real request
        checked += 1
        sleep(REQUEST_DELAY) # never hammer a service

    if response failed (404 / timeout / network error):
        bucket = NOT_FOUND_OR_UNAVAILABLE # dropped, never guessed in
    else:
        relevant_count = count postings whose title matches `keywords`
        if relevant_count > 0:
            bucket = VERIFIED_RELEVANT # eligible for promotion
        else:
            bucket = VERIFIED_NO_RELEVANT_MATCH # real board, nothing to add yet

if --apply:
    for each connector's VERIFIED_RELEVANT identifiers:
        append to discovery_sources.json[connector] if not already present
    (every other key in discovery_sources.json -- comments, Workday config,
    SmartRecruiters notes -- is copied through untouched)
```

Why relevance gates promotion, not just existence

A company can run a perfectly real Greenhouse/Lever/Ashby board and simply have zero Product/Business-Analysis/Project-Program openings today (e.g. an infra company hiring only engineers). Gating promotion on `relevant\_count > 0` at verification time keeps the maintained source list useful instead of growing unboundedly with boards that will never contribute a matching posting. This is a company-selection heuristic only -- it never filters at the JOB level; once a company is promoted, Stage 1 still fetches and keeps every posting from it (high recall), and Stage 3 alone decides individual job fit.

A note on "pagination" here

Greenhouse's and Ashby's public list endpoints are confirmed (by live request) to return a company's ENTIRE board in one response -- there is no page/offset parameter to iterate. verify_greenhouse/verify_lever/verify_ashby therefore each make exactly one HTTP request per candidate; MAX_PAGES_PER_COMPANY exists in config for shape-parity with Workday (which genuinely paginates, see Section 3.5) but is a no-op for these three.

3. Stage 1 -- Job Discovery & Normalization

3.1 Title expansion (search breadth)

The candidate's own target/secondary titles are expanded with curated, same-family variants before they're used as a search/filter keyword list -- broadening recall without drifting into unrelated job families.

```
expand_target_titles(target_titles, secondary_titles):
    base_words = union of words across all candidate titles
    for family, trigger_words in {product, business_analysis, project}:
        if base_words intersects trigger_words:
            add every curated variant for that family (e.g. 'product' triggers
              'Associate Product Manager', 'APM', 'Product Operations Analyst', ...)
    return expanded list # never adds an unrelated family like engineering/sales
```

3.2 Per-connector fetch + normalize

Each connector (Greenhouse, Lever, Ashby, SmartRecruiters, Workday) has its own fetch_*/normalize_* pair, but every one converges on the SAME output schema (Section 3.3) so downstream code never branches on source.

Connector	Fetch shape	Notes
Greenhouse	1 GET, full board	boards-api.greenhouse.io/v1/boards/{token}/jobs?content=true
Lever	1 GET, full board	api.lever.co/v0/postings/{slug}?mode=json
Ashby	1 GET, full board	api.ashbyhq.com/posting-api/job-board/{board}
SmartRecruiters	Paged GET (offset/limit)	Only companies with the public feed opted in
Workday	POST search, paged; GET per-job detail	cxs API -- same data the careers page itself fetches

3.3 Relevance filter applied during collection (still high recall)

```
title_is_relevant(title, keywords):
    title_words = normalized word set of the posting's title
    return True if ANY keyword phrase's full (normalized) word set
        is a SUBSET of title_words # every word of the phrase must appear
                                   # (stricter than 'any shared word' --
                                   # avoids 100s of false positives like
                                   # 'Manager -- Accounts' matching 'manager')
```

```
location_is_relevant(location, preferred_locations):
    if location is missing -> True # unknown is never rejected
    if 'remote'/'wfh' in location -> True # explicit preference
    else -> True only if it contains a preferred city
```

3.4 Common normalized schema (every connector converges here)

```
id, company, title, location, work_mode, industry, company_size,  
salary_min/max/currency, min/max_experience_years,  
required_skills[], preferred_skills[], description (<=4000 chars, HTML stripped),  
application_url, source_url, source_job_id, source, posted_date,  
date_discovered, discovered_at    -- filled by the connector at fetch time  
first_seen_at, last_seen_at       -- filled by the DEDUP step, Section 4
```

Fields the connector cannot determine (e.g. Ashby's list endpoint has no structured skills bullets, SmartRecruiters' list endpoint has no full job body) are left as empty/None rather than guessed -- this is a hard rule: nothing here is ever invented to fill a gap.

3.5 Workday connector detail (the one genuinely paginated source)

```
for each expanded keyword:  
  page = 0  
  while page < MAX_PAGES_PER_SOURCE:  
    results = POST /jobs {searchText: keyword, limit=20, offset=page*20}  
    collect externalPath of every result passing title/location relevance  
    page += 1  
    stop if this page returned fewer than 20 (no more pages)  
  fetch full detail (GET) for each unique externalPath found above  
  normalize each detail response into the common schema
```

4. Stage 2 -- Deduplication & History Merge

Every newly fetched posting is checked against BOTH this run's own results and the entire existing data/jobs.json using a confidence-tiered identity check -- any ONE tier matching is enough to call two postings the same job.

```

job_identity_keys(job):                                # strongest signal first
    1. (source, source_job_id)                        -- exact platform-native ID match
    2. canonical_application_url(url)                  -- same link, tracking params stripped,
                                                         host lowercased, no trailing slash
    3. (company, title, location, sorted(required_skills))
                                                         -- last resort; DELIBERATELY does NOT
                                                         collapse two same-titled postings at the
                                                         same company/location if their required
                                                         skills differ (two real, separate reqs)

identity_index = build from ALL existing jobs.json entries

for each new job:
    match = look up any of its 3 identity keys in identity_index
    if match found:
        duplicate += 1
        if match.last_seen_at != today: match.last_seen_at = today    # rediscovered
        # match.first_seen_at is NEVER touched
    else:
        job.first_seen_at = job.last_seen_at = today
        register all 3 of job's identity keys into identity_index
        new_unique += 1

save_jobs(path, existing + unique_new)                # APPEND -- history is never replaced

```

Why this matters

- A job can be discovered via two different sources (e.g. mirrored on both Greenhouse and a company's own career page indexed elsewhere) -- the canonical-URL tier catches that without needing an exact ID match.
- Re-running discovery daily never re-scores or duplicates unchanged postings -- it only refreshes last_seen_at, which the dashboard's "today" filter and staleness signal both depend on.
- Pre-existing jobs missing first_seen_at/last_seen_at (saved before those fields existed) are backfilled once, from date_discovered -- never invented from nothing.

5. Stage 3 -- V2 Matching Engine (score_job.py)

The core algorithm. It answers two SEPARATE questions for every posting: (a) how good a match is this, numerically, out of 100? and (b) should you actually apply? A high answer to (a) never auto-answers (b) -- (b) is computed independently by hard gates that can override any score.

5.1 Step A -- Role family classification

Classifies the posting into one of: PRODUCT, BUSINESS_ANALYSIS, PROJECT_PROGRAM, OTHER_TECH, SALES, LEGAL, MARKETING, HR, or UNKNOWN. Runs in priority order so an unambiguous phrase always wins over a generic word that happens to co-occur.

```

classify_role_family(title, description):
    1. Check UNAMBIGUOUS PHRASES in the title first:
        'counsel'/'attorney' -> LEGAL      (wins even if 'manager' also appears --
                                           e.g. 'Product Counsel Associate Manager' is LEGAL)
        recruiter/HR phrases -> HR        'marketing' -> MARKETING
        sales phrases         -> SALES     'software engineer' etc. -> OTHER_TECH
    2. Else, check TITLE WORD PATTERNS (order-independent word sets):
        {product} + {analyst|manager|owner} -> PRODUCT
        {business} + {analyst}              -> BUSINESS_ANALYSIS
        {functional|systems} + {analyst}     -> BUSINESS_ANALYSIS
        {project|program} + {manager|coordinator} -> PROJECT_PROGRAM
    3. Else, re-run step 1's phrase check against title + first 300 chars of
        description (last resort only -- avoids an incidental word deep in a
        long description overriding a genuinely clear but pattern-less title)
    4. Else -> UNKNOWN

candidate_target_role_families = { classify_role_family(t) for t in candidate's
                                   own target_titles + secondary_titles }

```

Family dampening -- how a mismatch suppresses two other components

```

if job's family is NOT one of the candidate's target families: dampening = 0.20
elif job's family is UNKNOWN (couldn't classify):                dampening = 0.60
else (exact family match):                                       dampening = 1.00
    -> capability_score and evidence_score (Section 5.3) are BOTH multiplied by
        this factor, so generic transferable skills can nudge but never rescue an
        unrelated role's score.

```

5.2 Step B -- Career level classification

Title wording is checked FIRST (most reliable), years of experience only as a fallback when the title gives no seniority signal at all.

```

classify_career_level(title, min_experience_years):
    'chief'/'ceo'/'vp'/'...' -> EXECUTIVE
    'director' / 'head of' / 'associate director' -> DIRECTOR
    'senior'/'sr'/'principal'/'staff' -> SENIOR
    'lead' / 'group product' / 'group manager' -> LEAD
    'intern'/'internship' -> INTERN
    'entry level'/'fresher'/'trainee' -> ENTRY_LEVEL
    'associate'/'junior'/'jr' -> JUNIOR
    bare 'manager' (and NOT a function-naming phrase
        like 'product manager'/'project manager') -> MANAGER
    -- else, if no keyword matched, fall back to years:
    <1y -> ENTRY_LEVEL    1-2y -> JUNIOR    2-4y -> MID_LEVEL
    4-7y -> SENIOR        >7y -> LEAD      no years at all -> UNKNOWN

```

5.3 Step C -- Nine weighted score components (sum to 100)

Role family match		20 pts
Capability relevance		20 pts
Relevant evidence		15 pts
Title alignment		10 pts
Experience fit		10 pts
Career level fit		10 pts
Location / work mode		8 pts
Industry / company size		4 pts
Salary fit		3 pts

Role family score (20 pts)

```

target family match -> 20          unclassifiable (UNKNOWN) -> 7 (35%)
not a target family -> 0          (and flags is_mismatch=True for the gates below)

```

Capability relevance (20 pts, dampened) -- weighted need-coverage

```

needs = {primary: [...], secondary: [...]} # fixed per role family, e.g. PRODUCT needs
                                             # primary=[product_management, analytics],
                                             # secondary=[stakeholder_comm, docs, ...]

strength(capability) = 1.0 professional | 0.65 practical | 0.0 none
weighted_sum = sum(strength(k)*2 for k in primary) + sum(strength(k) for k in secondary)
total_weight = 2*len(primary) + len(secondary)
raw_score = 20 * (weighted_sum / total_weight)
final = round(raw_score * family_dampening_factor) # Section 5.1

```

Relevant evidence (15 pts, dampened) -- phrase-coverage of required/nice-to-have skills

```

phrase_coverage(phrase, candidate_evidence_blob):
    words = significant (4+ letter, non-stopword) words in the phrase
    return (# of those words found in blob) / (total words) # 1.0 if phrase has no
                                                             significant words

required_raw = 12 * mean(coverage(p) for p in required_skills) (12 if none required)
nice_raw      = 3 * mean(coverage(p) for p in nice_to_have)    (3 if none listed)
final = round((required_raw + nice_raw) * family_dampening_factor)

```

Title alignment (10 pts) -- literal title text, independent of family

```

exact/substring match of an own TARGET title      -> 10   (100%)
exact/substring match of an own SECONDARY title   -> 7    (70%)
different title, but same role family as a target  -> 5    (50%)
no family match, but some title words overlap known ones -> up to 3, scaled by overlap
no relation at all                                -> 0

```

Experience fit (10 pts) -- gradient, senior-conservative

```

no min/max experience stated in posting           -> 10   (never penalize unknown)
underqualified (years < min_required), gap =
  <=1y -> 85%   <=2y -> 60%   <=3y -> 40%   <=5y -> 20%   >5y -> 5%
  of the 10pt weight -- and if the role reads SENIOR+, that fraction is further
  multiplied by 0.4 (a senior role's experience bar is not softened)
overqualified (years > max_required) -> max(70%, 100% - 5%*years_over) of weight
meets stated range                             -> 10   (100%)

```

Career level fit (10 pts)

```

ENTRY_LEVEL / JUNIOR -> 10 (100%)      MID_LEVEL -> 6 (60%)
UNKNOWN -> 5 (50%)                     SENIOR/LEAD/MANAGER/DIRECTOR/EXECUTIVE -> 1 (10%)

```

Location / work mode (8 pts = 4 + 4), Industry (4 pts = 3+1), Salary (3 pts)

```

Common rule across all three: a MISSING value never costs points --
it is scored as if it matched. Only a value that is STATED and clearly outside
preference costs points (down to 0 for work mode/location, avoided industry;
down to 50% for a non-preferred-but-not-avoided industry or company size).
Salary: >= target -> 3 (100%); between candidate's min and target -> 1.5-3, linear;
        below candidate's minimum acceptable -> 0 (and flags critical_missing).

```

5.4 Total score & critical-missing flag

```

total = clamp(0, 100, round(sum of all 9 component scores))

```

```

critical_missing = True if ANY of:

```

- industry is on the candidate's avoid list
- salary is stated and below the candidate's stated minimum
- MOST required skills show near-zero evidence coverage
 - (>= half of required_skills are 'weak matches' AND overall required-evidence raw score is below 30% of its own max)

6. Step D -- Application Decision (independent of the numeric score)

decide_application() runs AFTER total_score is final, but its gates are checked in a fixed order and any one of the first four can force SKIP regardless of how high total_score is.

```
level = career_level ; gap = experience_gap_years ; is_senior_plus = level in
    {SENIOR, LEAD, MANAGER, DIRECTOR, EXECUTIVE}

1. is_senior_plus?                -> SKIP ('above your target level')
2. role_family mismatch?          -> SKIP ('role family mismatch')
3. experience gap > 2 years?       -> SKIP ('exceeds 2-year threshold')
4. critical_missing?              -> SKIP ('avoided industry / below-min
                                   salary / near-total skills gap')
5. total_score < 75?              -> SKIP ('below review threshold')

-- only postings surviving ALL FIVE gates above reach this point --

6. ALL of: total_score >= 85
    AND not a family mismatch
    AND level in {ENTRY_LEVEL, JUNIOR}
    AND gap <= 1 year
    AND not critical_missing
    AND appears_legitimate (has a real http(s) apply URL AND description > 50 chars)
? -> AUTO_APPLY

7. otherwise                      -> REVIEW (worth a human look; reason
                                           lists which soft condition
                                           fell short of AUTO_APPLY)
```

Why score and decision are split

A posting can score very high (strong capability/evidence match, great location/salary fit) and still be SKIP -- most commonly because the title reads Senior/Staff/Principal/Director at a large global company. That gate exists specifically so a numerically attractive but seniority-mismatched role is never auto-applied to, or even offered as a REVIEW candidate the candidate would have to manually reject each time.

7. Worked Example (from a live run)

"Product Manager, Developer Ecosystem" @ Reddit (Remote - United States), discovered via the Greenhouse connector, scored 91/100 and was routed to REVIEW.

Component	Score	Why
Role family (20)	20	Title pattern {product}+{manager} -> PRODUCT, a target family
Capability (20, dampened x1.0)	~16	Professional product-adjacent evidence, not a perfect need match
Evidence (15, dampened x1.0)	~11	Partial phrase coverage of the listed required skills
Title alignment (10)	5	Same family as a target title, not an exact/secondary match
Experience fit (10)	~2	Senior-flavored posting scored conservatively despite no explicit years
Career level fit (10)	1	Title reads senior-leaning at a large company -> only 10% credit
Location/work mode (8)	8	Remote, matches an open work-mode preference
Industry (4)	4	Not on the avoid list
Salary (3)	3	Not stated -- never penalized

Decision walk: not senior+ by the STRICT title-marker check (no 'Senior/Staff/Principal' token in this exact title) -> passes gate 1. Family matches -> passes gate 2. No stated experience gap -> passes gate 3. No critical missing requirement -> passes gate 4. Score 91 >= 75 -> passes gate 5. AUTO_APPLY needs level in {ENTRY_LEVEL, JUNIOR} -- this title's implied level is MID/SENIOR-leaning, so gate 6 fails -> falls through to REVIEW.

8. Determinism, Idempotency & No-LLM Guarantee

- Every function above is pure, rule-based Python -- same input always produces the same output. No network call, randomness, or model inference happens inside `score_job.py`.
- Discovery + dedup + normalization + scoring are all deterministic code (Section 1); nothing sends a job description to an LLM at any stage of this pipeline.
- Re-scoring an unchanged job produces byte-identical output -- `score_and_annotate()` only needs to run once per newly-discovered job; rediscovered duplicates are never re-scored, only their `last_seen_at` is refreshed.
- The dashboard (`dashboard.py`) is a pure read/filter/sort layer over the already-scored `data/jobs.json` -- it never re-invokes any of the algorithms in this document.